

ARTIFICIAL INTELLIGENCE

INTERNSHIP

NAME : SHREYA

1.CHATBOT WITH RULE-BASED RESPONSES

Build a simple chatbot that responds to user inputs based on predefined rules. Use if-else statements or pattern matching techniques to identify user queries and provide appropriate responses. This will give you a basic understanding of natural language processing and conversation flow.

```
print("Chatbot: Hello! I am your simple chatbot. Type 'bye' to exit.")
```

```
while True:
```

```
    user = input("You: ").lower() # convert input to lowercase
```

```
    if user == "hello" or user == "hi":
```

```
        print("Chatbot: Hi there! How can I help you?")
```

```
    elif "how are you" in user:
```

```
        print("Chatbot: I'm just code, but I'm doing great! 😊")
```

```
    elif "your name" in user:
```

```
        print("Chatbot: I am a simple rule-based chatbot.")
```

elif "help" in user:

print("Chatbot: I can answer basic questions like greetings and simple info.")

elif "bye" in user:

print("Chatbot: Goodbye! Have a nice day 🤖")

break

else:

print("Chatbot: Sorry, I don't understand that yet.")

output:

Chatbot: Hello! I am your simple chatbot. Type 'bye' to exit.

You: hello

Chatbot: Hi there! How can I help you?

You: what is your name

Chatbot: I am a simple rule-based chatbot.

You: how are you

Chatbot: I'm just code, but I'm doing great! 😊

You: help

Chatbot: I can answer basic questions like greetings and simple info.

You: tell me joke

Chatbot: Sorry, I don't understand that yet.

You: bye

Chatbot: Goodbye! Have a nice day 🤝

TIC-TAC-TOE AI

2. Implement an AI agent that plays the classic game of Tic-Tac-Toe against a human player. You can use algorithms like Minimax with or without Alpha-Beta Pruning to make the AI player unbeatable. This project will help you understand game theory and basic search algorithms.

```
import math
```

```
board = [" " for _ in range(9)]
```

```
def print_board():
```

```
    print()
```

```
    print(board[0] + " | " + board[1] + " | " + board[2])
```

```
    print("---+---+---")
```

```
    print(board[3] + " | " + board[4] + " | " + board[5])
```

```
    print("---+---+---")
```

```
    print(board[6] + " | " + board[7] + " | " + board[8])
```

```
    print()
```

```
def check_winner(b, player):
```

```
    win_positions = [
```

```
        [0,1,2],[3,4,5],[6,7,8],
```

```
        [0,3,6],[1,4,7],[2,5,8],
```

```
        [0,4,8],[2,4,6]
```

```
    ]
```

```
    return any(all(b[i] == player for i in pos) for pos in win_positions)
```

```
def is_full(b):
```

```
    return " " not in b
```

```
def minimax(b, depth, is_max):
```

```
    if check_winner(b, "O"):
```

```
        return 1
```

```
    if check_winner(b, "X"):
```

```
        return -1
```

```
    if is_full(b):
```

```
        return 0
```

```
    if is_max:
```

```
        best = -math.inf
```

```
        for i in range(9):
```

```
            if b[i] == " ":
```

```
                b[i] = "O"
```

```
                best = max(best, minimax(b, depth+1, False))
```

```

def check_winner(b, player):
    win_positions = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    return any(all(b[i] == player for i in pos) for pos in win_positions)

```

```

def is_full(b):
    return " " not in b

```

```

def minimax(b, depth, is_max):
    if check_winner(b, "O"):
        return 1
    if check_winner(b, "X"):
        return -1
    if is_full(b):
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == " ":
                b[i] = "O"
                best = max(best, minimax(b, depth+1, False))

```

```
b[i] = " "
```

```
return best
```

```
else:
```

```
best = math.inf
```

```
for i in range(9):
```

```
    if b[i] == " ":
```

```
        b[i] = "X"
```

```
        best = min(best, minimax(b, depth+1, True))
```

```
    b[i] = " "
```

```
return best
```

```
def ai_move():
```

```
    best_score = -math.inf
```

```
    move = 0
```

```
    for i in range(9):
```

```
        if board[i] == " ":
```

```
            board[i] = "O"
```

```
            score = minimax(board, 0, False)
```

```
            board[i] = " "
```

```
            if score > best_score:
```

```
                best_score = score
```

```
                move = i
```

```
    board[move] = "O"
```

```
def player_move():
```

```
    move = int(input("Enter position (1-9): ")) - 1
```

```
if board[move] == " ":
```

```
    board[move] = "X"
```

```
else:
```

```
    print("Invalid move!")
```

```
    player_move()
```

```
print("Tic-Tac-Toe: You (X) vs AI (O)")
```

```
print_board()
```

```
while True:
```

```
    player_move()
```

```
    print_board()
```

```
    if check_winner(board, "X"):
```

```
        print("You win!")
```

```
        break
```

```
    if is_full(board):
```

```
        print("It's a draw!")
```

```
        break
```

```
    print("AI's move...")
```

```
    ai_move()
```

```
    print_board()
```

```
    if check_winner(board, "O"):
```

```
        print("AI wins!")
```

```
        break
```

```
    if is_full(board):
```

```
print("It's a draw!")
```

```
break
```

output:

Tic-Tac-Toe: You (X) vs AI (O)

```
| | |  
--+--+--  
  
| | |  
--+--+--  
  
| | |
```

Enter position (1-9): 5

```
| | |  
--+--+--  
  
| X |  
--+--+--  
  
| | |
```

AI's move...

```
| O |  
--+--+--  
  
| X |  
--+--+--  
  
| | |
```

Enter position (1-9): 1

```
X | O |
```

--+--+--

| X |

--+--+--

| |

AI's move...

X | O | O

--+--+--

| X |

--+--+--

| |

Enter position (1-9): 9

X | O | O

--+--+--

| X |

--+--+--

| | X

AI's move...

X | O | O

--+--+--

O | X |

--+--+--

| | X

AI wins!

I

RECOMMENDATION SYSTEM

3. Create a simple recommendation system that suggests items to users based on their preferences. You can use techniques like collaborative filtering or content-based filtering to recommend movies, books, or products to users.

```
import numpy as np

from sklearn.metrics.pairwise import cosine_similarity

users = ["Ram", "Sita", "John", "Meena"]
movies = ["Movie A", "Movie B", "Movie C", "Movie D"]

ratings = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [0, 0, 5, 4]
])

similarity = cosine_similarity(ratings)

def recommend(user_index):
    print(f"\nRecommendations for {users[user_index]}:")
```

```
sim_scores = list(enumerate(similarity[user_index]))
```

```
sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
```

```
similar_user = sim_scores[1][0]
```

```
print("Most similar user:", users[similar_user])
```

```
for i in range(len(movies)):
```

```
    if ratings[user_index][i] == 0 and ratings[similar_user][i] > 3:
```

```
        print("Recommended:", movies[i])
```

```
recommend(0)
```

output:

Recommendations for Ram:

Most similar user: Sita